

HelixCode

HelixCode

वितरित AI विकास मंच जो कार्य को विभाजित करता है, उसे संरक्षित रखता है और आपकी जगह कभी नहीं खोता।

सारांश

HelixCode एक एंटरप्राइज़-ग्रेड, Go-आधारित वितरित AI विकास मंच है जो विकास कार्यों को SSH-प्रबंधित श्रमिकों के नेटवर्क में बुद्धिमानी से विभाजित करता है। इसमें स्वचालित चेकपॉइंटिंग और रोलबैक की सुविधा है, जिससे कोई भी कार्य कभी नहीं खोता। यह बहु-प्रदाता LLM एकीकरण, पूर्ण विकास-जीवनचक्र वर्कफ़्लो और क्रॉस-प्लेटफ़ॉर्म डिलीवरी को REST, CLI, TUI और MCP इंटरफ़ेस के पीछे एकीकृत करता है।

संक्षिप्त विवरण

HelixCode एक वितरित AI विकास मंच है जिसे Go में लिखा गया है। यह कार्य को SSH-आधारित श्रमिक नेटवर्क में बुद्धिमान कार्यों में विभाजित करता है, स्वचालित चेकपॉइंटिंग और रोलबैक के साथ प्रगति को संरक्षित रखता है, कई LLM प्रदाताओं को एकीकृत करता है, और REST, CLI, TUI तथा MCP इंटरफ़ेस के माध्यम से संपूर्ण विकास जीवनचक्र को संचालित करता है।

विस्तृत विवरण

HelixCode एक एंटरप्राइज़-ग्रेड वितरित AI विकास मंच (`dev.helix.code`, MIT) है, जो अपने टैगलाइन के वादे को शब्दशः पूरा करता है: कार्य को विभाजित करो, उसे संरक्षित रखो, और अपनी जगह कभी मत खोना। इसे बुद्धिमान कार्य विभाजन, स्वचालित कार्य संरक्षण और क्रॉस-प्लेटफ़ॉर्म विकास वर्कफ़्लो के लिए डिज़ाइन किया गया है। यह Go में लिखा गया है, जो वितरित कंप्यूटिंग की माँगों—समानांतरता और एकल-बाइनरी पोर्टेबिलिटी—को पूरा करता है। इसमें स्वचालित चेकपॉइंटिंग, रोलबैक और रीयल-टाइम मॉनिटरिंग को मूलभूत सुविधाओं के रूप में शामिल किया गया है, न कि वैकल्पिक ऐड-ऑन के रूप में।

इसकी वास्तुकला REST + WebSocket + MCP API सतह को केंद्रित कोर सेवाओं के समूह पर आधारित करती है—JWT प्रमाणीकरण और सत्र प्रबंधन, SSH-आधारित श्रमिक-पूल प्रबंधन (स्वास्थ्य मॉनिटरिंग सहित), कार्य प्रबंधन (चेकपॉइंटिंग और निर्भरता प्रबंधन के साथ), परियोजना और वर्कफ़्लो प्रबंधन, तथा एकीकृत LLM प्रदाता परत—सभी

PostgreSQL पर संग्रहीत, जिसमें Redis को वैकल्पिक समन्वय और कैशिंग स्तर के रूप में उपलब्ध कराया गया है। वितरित श्रमिक स्वचालित रूप से नेटवर्क पर स्थापित हो जाते हैं, इसलिए श्रमिकों के समूह को बढ़ाना केवल सर्वर को किसी मशीन की ओर निर्देशित करने का मामला है, न कि मैनुअल प्रोविज़निंग का। बहु-क्लाइंट इंटरफ़ेस CLI, टर्मिनल UI, REST और मोबाइल फ्रेमवर्क को समाहित करते हैं, ताकि एक ही मंच को स्क्रिप्ट, टर्मिनल या ऐप से एक्सेस किया जा सके।

HelixCode संपूर्ण विकास जीवनचक्र को शुरू से अंत तक संचालित करता है: योजना, निर्माण, परीक्षण और रिफैक्टरिंग वर्कफ़्लो स्वचालित रूप से निर्भरता जागरूकता और बहु-सत्र संदर्भ ट्रैकिंग के साथ निष्पादित होते हैं, ताकि लंबे समय तक चलने वाले प्रयासों का तारतम्य बाधाओं और मशीन सीमाओं के पार बना रहे। यह कई LLM प्रदाताओं—llama.cpp, Ollama और OpenAI—को एक इंटरफ़ेस के पीछे एकीकृत करता है, फिर हार्डवेयर-जागरूक मॉडल चयन जोड़ता है जो उपलब्ध CPU/GPU/मेमोरी का पता लगाता है और मॉडल को मशीन के अनुरूप बनाता है। यह उन समस्याओं के लिए उन्नत तर्क रणनीतियों जैसे चेन-ऑफ़-थॉट और ट्री-ऑफ़-थॉट्स का भी समर्थन करता है जिन्हें एकल प्रयास से अधिक की आवश्यकता होती है। Model Context Protocol को कई ट्रांसपोर्ट पर लागू किया गया है ताकि उपकरणों और संदर्भों का मानकीकृत आदान-प्रदान हो सके और बहु-चैनल सूचनाएँ (Slack, Discord, ईमेल, Telegram) टीमों को वितरित कार्य की प्रगति के बारे में अवगत रखती हैं। यह Linux, macOS, Windows, Aurora OS और SymphonyOS को लक्षित करता है।

सामग्री

हमने इसे क्यों बनाया

वितरित और AI-सहायित विकास में आमतौर पर तब संदर्भ और प्रगति खो जाती है जब कार्यों को अलग-अलग मशीनों पर बाँटा जाता है या बीच में रुकावट आती है। HelixCode को इस तरह से डिज़ाइन किया गया है कि कार्य विभाजन बुद्धिमानी से हो और काम का संरक्षण स्वचालित—ताकि एक बड़े विकास प्रयास को तोड़ा जा सके, कार्यकर्ता नेटवर्क में वितरित किया जा सके, चेकपॉइंट किया जा सके और स्थिति खोए बिना फिर से शुरू या वापस लाया जा सके।

यह क्यों गेम-चेंजर है

यह वितरित AI विकास को टिकाऊ बनाता है—वह क्षमता जो तब कभी व्यावहारिक नहीं थी जब टीमों इन टुकड़ों को हाथ से जोड़ती थीं। तीन चीज़ें जो आमतौर पर तीन अलग-अलग टूल्स में होती हैं, अब एक ही प्लेटफ़ॉर्म पर आ जाती हैं: वितरित कंप्यूटिंग (SSH कार्यकर्ता नेटवर्क जिसमें स्वचालित इंस्टॉलेशन और स्वास्थ्य निगरानी शामिल है), AI विकास सहायता (बहु-प्रदाता एलएलएम्स जिसमें तर्क और टूल कॉलिंग की सुविधा है), और पूर्ण जीवनचक्र वर्कफ़्लो स्वचालन। इन सबको जोड़ने वाली कड़ी है डेटाबेस-समर्थित चेकपॉइंटिंग: चूंकि कार्य स्थिति, चेकपॉइंट्स और निर्भरताएँ PostgreSQL में संरक्षित रहती हैं, इसलिए एक ऐसा काम जो कई मशीनों और कई सत्रों में फैला हो, उसे ठीक उसी जगह से वापस लाया या फिर से शुरू किया जा सकता है जहाँ वह रुका था। रुकावटें और बंटा हुआ काम अब प्रगति के नुकसान का कारण नहीं रह जाते, बल्कि एक सामान्य, पुनर्प्राप्त होने योग्य घटना बन जाते हैं।

क्या है नवाचारी

- कार्य संरक्षण एक मूलभूत सिद्धांत के रूप में: वितरित विकास कार्यों पर स्वचालित चेकपॉइंटिंग और रोलबैक लागू करना, ताकि प्रगति रुकावट या मशीन विफलता के बावजूद सुरक्षित रहे, न कि उसके साथ ही गायब हो जाए।
- हार्डवेयर-जागरूक मॉडल चयन जो पता लगाए गए CPU/GPU/मेमोरी का निरीक्षण करता है और हर कार्य को उस मॉडल से जोड़ता है जिसे मशीन वास्तव में अच्छी तरह चला सके—प्रत्येक कार्यकर्ता के लिए मैनुअल ट्यूनिंग की ज़रूरत नहीं।
- एक प्लेटफॉर्म, पाँच प्रवेश द्वार: REST, WebSocket, CLI, TUI, और MCP, जहाँ MCP स्वयं कई ट्रांसपोर्ट पर उपलब्ध है ताकि टूल्स और एजेंट्स अपनी सुविधानुसार एकीकृत हो सकें।
- क्रॉस-प्लेटफॉर्म पहुँच जो सामान्य डेस्कटॉप तिकड़ी से आगे बढ़कर Aurora OS और SymphonyOS को भी शामिल करती है, जिससे कार्यकर्ता बेड़े का विस्तार उन प्लेटफॉर्म तक हो जाता है जिन्हें अधिकांश टूल्स नज़रअंदाज़ करते हैं।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- वितरित, रुकावट-सहनीय कार्यों में काम न खोना। जब कोई काम कई मशीनों पर बाँटा जाता है, तो किसी भी क्रैश या रुकावट से आमतौर पर जो भी प्रगति चल रही होती है, वह अधूरी रह जाती है। हमने कार्य को ही चेकपॉइंट्स और निर्भरताओं का वाहक बना दिया, जो PostgreSQL में संरक्षित रहते हैं, ताकि सिस्टम पिछली अच्छी स्थिति पर वापस जा सके या वहीं से फिर से शुरू कर सके—टिकाऊपन जो डेटा परत में निहित है, न कि नाज़ुक इन-मेमोरी स्थिति में।
- विविध कार्यकर्ता बेड़े का प्रबंधन। Linux, macOS, Windows, Aurora, और SymphonyOS मशीनों का नेटवर्क उपलब्धता और सेटअप के लिहाज़ से एक गतिशील लक्ष्य है। हम इसे एक समर्पित कार्यकर्ता-पूल सेवा के ज़रिए संभालते हैं जो SSH-आधारित पंजीकरण, नए नोड्स पर स्वचालित इंस्टॉलेशन, और निरंतर स्वास्थ्य निगरानी करती है, ताकि मशीनों के आते-जाते रहने पर भी बेड़ा ज्ञात और नियंत्रणीय बना रहे।
- प्रदाता और हार्डवेयर विविधता। LLM बैकएंड्स और उन्हें चलाने वाली मशीनें क्षमता में बहुत भिन्न होती हैं। हमने इसे एक एकीकृत LLM प्रदाता इंटरफ़ेस के पीछे छिपा दिया और इसे हार्डवेयर पहचान (CPU/GPU/मेमोरी) के साथ जोड़ा, जो बुद्धिमान मॉडल चयन को संचालित करता है, ताकि सही मॉडल सही मशीन पर पहुँचे—बिना यह सोचे कि कॉलर को दोनों में से किसी के बारे में तर्क करना पड़े।

सामग्री

तकनीकी स्टैक

- **Go (1.26+ आंतरिक मॉड्यूल)** — चुना गया क्योंकि इसकी गोरूटीन-आधारित समवर्तीता और एकल-बाइनरी आउटपुट ठीक वही है जो एक वितरित कार्यकर्ता प्रणाली को चाहिए: ऑर्किस्ट्रेशन के लिए सस्ता समानांतरवाद और एक स्व-निहित बाइनरी जो किसी भी नोड पर स्वतः स्थापित हो जाती है। इसमें सभी मुख्य सेवाएँ और CLI/सर्वर बाइनरी शामिल हैं।
- **Gin (HTTP फ्रेमवर्क)** — चुना गया एक तेज़, न्यूनतम REST परत के लिए जिसमें कम ओवरहेड हो; यह /api/v1 इंटरफ़ेस (प्रमाणीकरण, कार्यकर्ता, कार्य, परियोजनाएँ) प्रदान करता है जिससे हर क्लाइंट संवाद करता है।

- **PostgreSQL 15+ (pgx/v5 के माध्यम से)** — चुना गया स्थायी रिकॉर्ड प्रणाली के रूप में क्योंकि चेकपॉइंटिंग और रोलबैक के लिए लेन-देन संबंधी दृढ़ता आवश्यक है; इसमें 11-तालिकाओं वाला वितरित-कंप्यूटिंग स्कीमा (उपयोगकर्ता, कार्यकर्ता, कार्य, परियोजनाएँ, सत्र, llm_प्रदाता, सूचनाएँ) संग्रहीत है जो कार्य संरक्षण को संभव बनाता है।
- **Redis 7+ (वैकल्पिक, go-redis/v9)** — चुना गया एक वैकल्पिक कैशिंग और समन्वय स्तर के रूप में जो गर्म पथों को तेज़ करता है बिना किसी कड़ी निर्भरता के, ताकि न्यूनतम तैनाती केवल पोस्टग्रेस पर ही चले।
- **SSH** — चुना गया कार्यकर्ता-नियंत्रण परिवहन के रूप में क्योंकि यह पहले से ही हर जगह मौजूद है और पहले से ही सुरक्षित है; यह पूरे पूल में कार्यकर्ता पंजीकरण, स्वतः-स्थापना और दूरस्थ कमांड निष्पादन को संचालित करता है, बिना किसी विशेष एजेंट को पहले तैनात करने की आवश्यकता के।
- **Model Context Protocol (MCP)** — चुना गया मानकीकृत उपकरण और संदर्भ विनिमय के लिए ताकि बाहरी उपकरण और एजेंट एक खुले प्रोटोकॉल के माध्यम से एकीकृत हो सकें; इसे बहु-परिवहन समर्थन के साथ लागू किया गया है ताकि क्लाउंट जहाँ भी कनेक्ट हों, वहाँ उनसे मिला जा सके।
- **LLM प्रदाता (llama.cpp, Ollama, OpenAI)** — चुने गए स्थानीय और होस्टेड अनुमान को एकीकृत इंटरफ़ेस के पीछे समाहित करने के लिए, ताकि हार्डवेयर-जागरूक चयन किसी कार्य को स्थानीय मॉडल या होस्टेड मॉडल पर निर्देशित कर सके बिना कॉलर को अंतर पता चले।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। README में स्वयं “पूर्णतः पूर्ण / सभी 5 चरण” की स्थिति बताई गई है; यह पूर्णता परियोजना द्वारा घोषित है, स्वतंत्र रूप से सत्यापित नहीं, इसलिए पृष्ठ इसे बीटा के रूप में मानता है।
- उपरोक्त सभी विवरण रिपॉजिटरी के README से लिए गए हैं; विपणन संबंधी वाक्यांश (टैगलाइन) संपादकीय हैं, स्रोत मेट्रिक्स नहीं।

प्राथमिकता स्तर: Helix-प्राथमिक।